

Celebal Technologies Internship

Assignment Summary

Name: Anshita Garg

Internship: Celebal Technologies Data Engineering Internship

Week: 7

Delta Lake MERGE Implementation

Objective

The objective of this assignment was to understand and implement **incremental data processing** using **Delta Lake** in **Azure Databricks**. The task involved loading a CSV dataset into a Delta table, performing basic data cleaning, simulating incremental data, applying the Delta Lake **MERGE** operation, and validating the final output.

Environment Used

- **Platform:** Azure Databricks (Serverless)
 - **Language:** Python (PySpark)
 - **Framework:** Apache Spark 4.1.0
 - **Storage Format:** Delta Lake
 - **Dataset:** Superstore Dataset (Kaggle)
-

Tasks Performed

1. Data Loading

- Imported the Superstore CSV dataset into a PySpark DataFrame.
- Explored the dataset using `display()`, `printSchema()`, and row count functions.
- Verified successful data loading before further processing.

2. Data Cleaning

The dataset was analyzed for data quality.

- Checked for missing (null) values.
- Verified duplicate records.
- Renamed column names by replacing spaces with underscores to ensure compatibility with Delta Lake.
- Created a cleaned DataFrame for further processing.

Observation

- Missing Values: **0**
 - Duplicate Records: **0**
-

3. Delta Table Creation

The cleaned DataFrame was converted into a Delta Lake table using the Delta format.

```
df_clean.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("week7_databricks.assignment.customer_master_delta")
```

This enabled support for ACID transactions and incremental updates.

4. Incremental Data Simulation

An incremental dataset was prepared to simulate real-world incoming data.

The dataset contained:

- Existing customer records (to demonstrate **UPDATE**)
- New customer records (to demonstrate **INSERT**)

This mimics daily incremental data ingestion commonly used in enterprise data engineering workflows.

5. Delta Lake MERGE Operation

The Delta Lake **MERGE** command was used to synchronize the incremental dataset with the existing Delta table.

The MERGE operation performed:

- Updates on matching records (**WHEN MATCHED**)
- Inserts for new records (**WHEN NOT MATCHED**)

This demonstrated efficient handling of incremental data without reloading the complete dataset.

6. Validation

After the MERGE operation:

- Final row count was verified.
 - Duplicate records were checked.
 - Updated and newly inserted records were successfully validated.
-

Results

- ✓ Dataset successfully loaded into Delta Lake.
 - ✓ No missing values found.
 - ✓ No duplicate records found.
 - ✓ Incremental dataset created successfully.
 - ✓ Existing records updated successfully using MERGE.
 - ✓ New records inserted successfully using MERGE.
 - ✓ Final dataset validated with correct row count and no duplicate records.
-

Learning Outcomes

Through this assignment, the following concepts were learned:

- Working with PySpark DataFrames
 - Data cleaning using Spark
 - Delta Lake architecture
 - Delta Table creation
 - Incremental data processing
 - MERGE (Upsert) operation
 - Data validation techniques
 - Azure Databricks Serverless environment
 - Unity Catalog integration
-

Conclusion

This assignment demonstrated the implementation of an end-to-end incremental data processing workflow using Delta Lake in Azure Databricks. By leveraging the MERGE operation, both record updates and new record insertions were performed efficiently within a single transaction. The exercise highlighted the advantages of Delta Lake for maintaining reliable, scalable, and ACID-compliant data pipelines, making it a valuable technology for modern data engineering and analytics solutions.